



ELP

Sistema Integrado de Gestão de Obras

Manual Técnico Completo do Aplicativo Android



PLATAFORMA
Android Nativo



FRAMEWORK
React Native
Expo SDK 57



BACKEND
Railway
Flask + PostgreSQL

- ✓ Offline-First com SQLite local
- ✓ Sincronização automática com Railway
- ✓ Relatórios de Vistoria com fotos e checklist
- ✓ Relatório Express para campo
- ✓ Agenda de Visitas com cores por engenheiro
- ✓ Painel de Aprovações e Reembolsos
- ✓ Atualização OTA automática via Expo Updates

Sumário

1.1 Camadas do Sistema	4
1.2 Fluxo de Dados	4
2.1 Recuperação de Senha	5
3.1 KPIs — Cálculo em Tempo Real	6
3.2 Ações Rápidas	6
4.1 Campos do Cadastro	7
5.1 Etapas de Criação de um Relatório	8
5.2 Ciclo de Vida do Status	8
6.1 Diferenças do Relatório Padrão	10
6.2 Campos do Express	10
7.1 Auto-Baixa Automática (Lógica de Negócio)	11
8.1 Fórmula de Cálculo	12
8.2 Campos e Status	12
9.1 Fluxo de Aprovação	13
10.1 Módulo Contatos	14
10.2 Módulo Lembretes	14
10.3 Ajustes e Configurações	14
10.4 Gestão de Usuários (Admin)	14
11.1 Três Gatilhos Automáticos	15
11.2 Processo de Sync Completo	15
11.3 Tabela sync_queue	15
12.1 Tabelas Principais	17
12.2 Campo sync_status (todas as tabelas)	17
13.1 Ciclo de Deploy Automático	18
13.2 Build Local	18
14.1 Mobile App	19



14.2 Backend (Railway) 19

1. Visão Geral e Arquitetura

O **ELP Android** é um aplicativo nativo React Native desenvolvido em TypeScript sobre o framework **Expo SDK 57**, concebido com paradigma **Offline-First**. Todo dado criado pelo engenheiro em campo é gravado localmente em **SQLite** antes de qualquer comunicação com o servidor, garantindo funcionamento pleno mesmo sem internet.

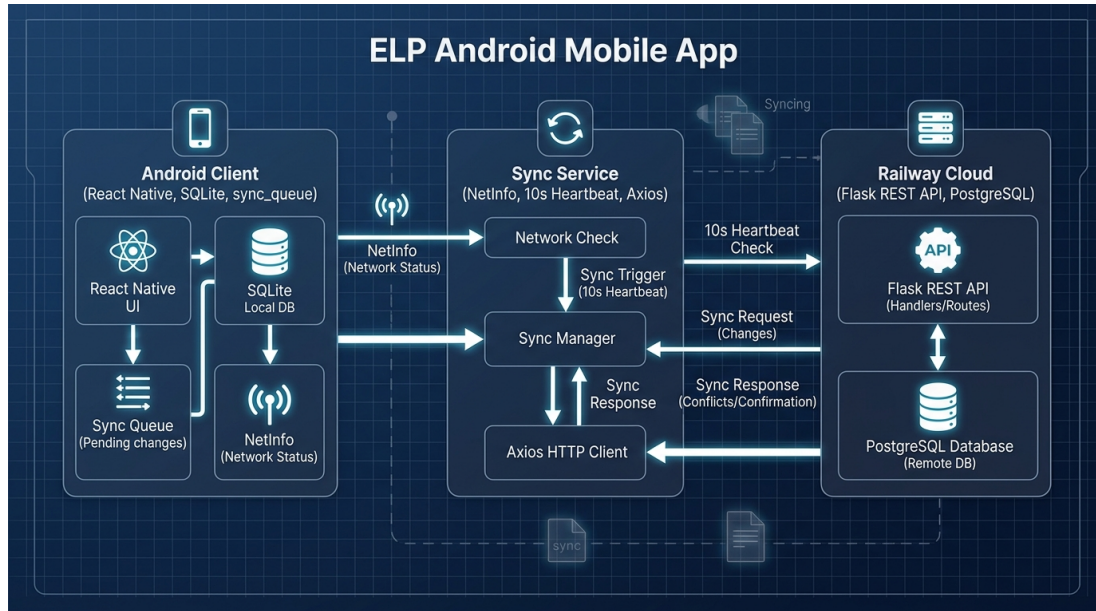


Diagrama de Arquitetura Offline-First — ELP Android (React Native + SQLite + Railway Cloud)

1.1 Camadas do Sistema

Camada	Tecnologia	Responsabilidade
App Mobile	React Native + Expo SDK 57	Interface, navegação, captura de fotos/GPS
Banco Local	expo-sqlite (SQLite nativo)	Persistência offline, fila de sincronização
Backend Nuvem	Flask/Python no Railway	API REST, PostgreSQL, geração de PDF, e-mail

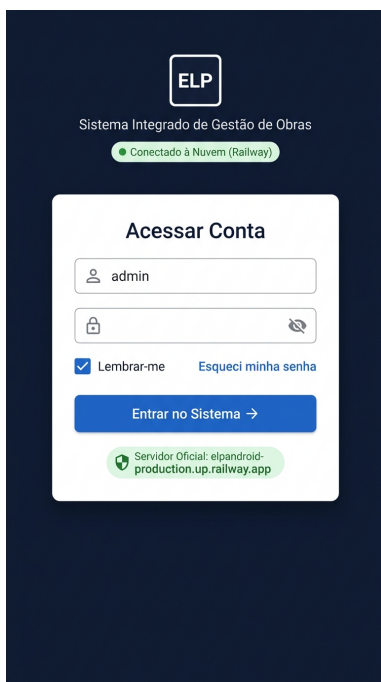
1.2 Fluxo de Dados

```
Ação do Usuário
↓
SQLite Local (imediato – funciona sem internet)
↓
sync_queue (enfileira a operação HTTP pendente)
↓
NetInfo detecta conexão?
■■■ SIM → HTTP POST/PUT para Railway → marca "synced"
■■■ NÃO → mantém "pending" → retry automático a cada 10s
```

URL de Produção: <https://elpandroid-production.up.railway.app>

2. Tela de Login

Arquivo: [src/screens/auth/LoginScreen.tsx](#) (550 linhas)



Passo 1 — Abertura do App

O AuthContext verifica AsyncStorage pela chave `obraflow_current_user`. Se sessão válida existir, redireciona diretamente para o Dashboard sem re-autenticação.

Passo 2 — Indicador de Conectividade

O pill no topo usa NetworkContext (via `@react-native-community/netinfo`) para exibir:

- **Conectado à Nuvem (Railway)** — verde
- **Modo Offline (Banco Local)** — âmbar

Passo 3 — Autenticação

- POST `/api/login` (principal)
- POST `/login` (fallback)
- GET `/api/current-user` (dados do perfil)

Modo Offline: valida usuário contra cache do AsyncStorage. Se usuário ≥ 3 chars e senha ≥ 4 chars, cria conta local automaticamente.

Lembrar-me: salva credenciais em AsyncStorage chave `@elp_remember_login`.

■ ■ Na primeira abertura, os campos já vêm preenchidos com `admin/admin` como padrão de desenvolvimento. Em produção, altere as credenciais imediatamente.

2.1 Recuperação de Senha

Toque em "**Esqueci minha senha**" abre Modal com campo de e-mail. Dispara POST `/api/forgot-password` com `{email, username}`. O backend envia e-mail com link de redefinição.

3. Dashboard Principal

Arquivo: [src/screens/dashboard/DashboardScreen.tsx](#) (1.346 linhas)



O Dashboard é 100% personalizável. A ordem e visibilidade de cada seção são salvas no AsyncStorage e persistidas entre sessões.

Seções disponíveis:

- Logo & Banner Principal
- Identificação do Usuário (nome + empresa)
- Aviso de Aprovações Pendentes (só aprovadores)
- Painel de Indicadores (4 KPI cards)
- Ações Rápidas no Canteiro
- Relatórios Recentes
- Próximas Visitas

Pull-to-Refresh: arraste para baixo para triggerSync() + recarregar dados do SQLite.

3.1 KPIs — Cálculo em Tempo Real

```
# Calculados a partir do banco SQLite local:
obrasAtivas = projetos.filter(p => p.status === 'Ativo').length
relatoriosPendentes = relatorios.filter(r => r.status === 'Aguardando Aprovação').length
visitasAgendadas = visitas.filter(v => v.status === 'Agendada').length
relatoriosRascunho = relatorios.filter(r => r.status === 'em_andamento').length
```

3.2 Ações Rápidas

Botão	Cor	Navega Para
Novo Relatório	#2563EB (Azul)	ReportFormScreen
Relatório Express	#059669 (Verde)	ExpressReportsScreen
Agendar Visita	#7C3AED (Roxo)	VisitFormScreen
Cadastrar Obra	#D97706 (Âmbar)	ProjectFormScreen
Gestão Usuários ★	#4F46E5 (Índigo)	UserManagementScreen (só admin)

4. Módulo Obras

Gerenciamento completo do portfólio de obras. Acessado pela aba **"Obras"** (ícone de prédio) na bottom tab bar.

4.1 Campos do Cadastro

Grupo	Campo	Observação
Identificação	Número da Obra	Ex: OBR-2026-001 (único)
Identificação	Nome da Obra	—
Identificação	Tipo de Obra	Edifício Residencial, Comercial, Industrial...
Identificação	Construtora / Cliente	—
Identificação	Responsável Técnico	Vinculado ao usuário
Identificação	E-mail Principal	Para notificações automáticas
Localização	Endereço	Texto + lat/long GPS
Status	Status	Ativo Concluído Suspenso
Status	Data Início / Previsão Fim	ISO 8601
Fachada	Elementos Construtivos Base	—
Fachada	Chapisco Colante e Regularização	—
Fachada	Argamassa de Emboço (traço)	Ex: 1:1:6 com aditivo
Fachada	Acabamentos e Revestimento	—
Fachada	Acabamento de Peitoris / Muretas	—
Fachada	Definição de Frisos e Cor	—
Fachada	Observações da Fachada	Campo livre

5. Módulo Relatórios de Vistoria

Arquivo: `src/screens/reports/ReportFormScreen.tsx` (1.176 linhas) — módulo mais complexo do aplicativo.

Funcionalidades-chave:

- 1. Rascunho Imediato:** ao selecionar uma obra, o rascunho é criado instantaneamente no SQLite e, se online, o número oficial é reservado no servidor (REL-XXXX-NNN).
- 2. Auto-Save:** debounce de 1 segundo monitora qualquer mudança nos campos e salva automaticamente no SQLite. Indicador 'Auto-salvo HH:MM:SS' no header.
- 3. Checklist Técnico:** 7 itens pré-definidos de fachada, cada um com checkbox e campo de observação individual.
- 4. Fotos:** câmera + galeria, anotações visuais, legendas pré-definidas por categoria.
- 5. UUID dual:** uuid + uuid_local para reconciliação offline/online sem conflitos.

5.1 Etapas de Criação de um Relatório

Etapa 1 — Inicialização do Rascunho	Ao selecionar o projeto, <code>initImmediateDraft()</code> é chamado. Se online, faz POST <code>/api/relatorios</code> e recebe o número oficial. Se offline, gera ID local e enfileira o POST na <code>sync_queue</code> para execução futura.
Etapa 2 — Auto-Save Contínuo	<code>useEffect</code> com <code>debounce</code> de 1s monitora <code>titulo</code> , <code>descricao</code> , <code>observacoesFinais</code> , <code>checklist</code> e <code>fotos</code> . Salva no SQLite a cada mudança. Impossível perder dados.
Etapa 3 — Preenchimento	Título, Categoria (Geral/Fachada/Estrutura/Alvenaria/Impermeabilização/Pintura/Acabamentos), Local, Descrição Técnica, Observações Finais.
Etapa 4 — Checklist	7 itens padrão ABNT de vistoria de fachada. Cada item tem checkbox (verde/vazio) e campo de observação textual expandível.
Etapa 5 — Fotos	Câmera (<code>expo-image-picker launchCameraAsync</code>) ou Galeria. Cada foto gera objeto com <code>uri_local + base64 + anotações_dados</code> (JSON das marcações). <code>PhotoEditorModal</code> permite desenhar sobre a foto.
Etapa 6 — Envio para Aprovação	Botão azul 'Enviar para Aprovação' → muda status para 'Aguardando Aprovação' → PUT <code>/api/relatorios/{id}</code> ou enfileira se offline → navega de volta para lista.

5.2 Ciclo de Vida do Status

em_andamento (Rascunho)



Aguardando Aprovação



Aprovado Rejeitado

6. Módulo Relatório Express

Modo simplificado e autônomo para inspeções rápidas em campo, sem necessidade de obra pré-cadastrada no sistema.

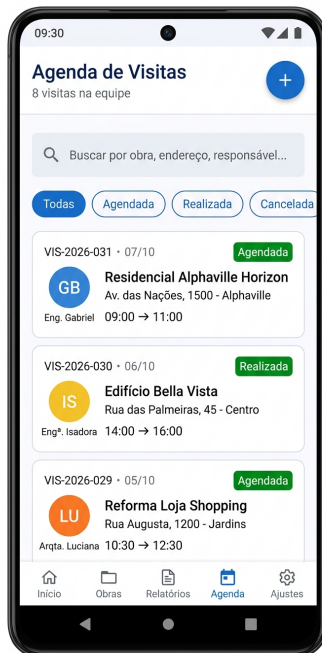
6.1 Diferenças do Relatório Padrão

Aspecto	Relatório Padrão	Relatório Express
Vínculo com obra	Obrigatório (FK SQLite)	Opcional (texto livre)
Número	Reservado no servidor (REL-...)	Gerado localmente (EXP-XXXX)
Tabela SQLite	relatorios	relatorios_express
PDF	Gerado pelo backend Railway	Gerado localmente via pdfService.ts
Foto	Múltiplas, com anotações	1 foto principal
GPS	Opcional	Botão de captura automática
Duplicar	Não disponível	Duplica com 1 toque

6.2 Campos do Express

- 1. Título (default: 'Relatório Express de Visita')
- 2. Número (EXP-XXXX gerado aleatoriamente)
- 3. Data da Visita (editável, default: hoje)
- 4. Cliente / Construtora
- 5. Nome da Obra (obrigatório)
- 6. Endereço / GPS (captura automática)
- 7. Acompanhantes (nomes dos presentes)
- 8. Informações Técnicas (seção colapsável)
- 9. Checklist (seção colapsável, texto livre)
- 10. Observações Gerais
- 11. Foto principal

7. Módulo Agenda de Visitas



Sistema de Cores por Engenheiro

Cada membro da equipe tem cor fixa no calendário:

- Leopoldo — Azul #1E88E5
- Mateus — Verde #43A047
- Isadora — Amarelo #FDD835
- Luciana — Laranja #FB8C00

Filtros disponíveis:

Todas | Agendada | Realizada | Cancelada

Busca Full-Text:

Filtra por projeto_nome, projeto_outros, observacoes, endereco_gps, responsavel_nome.

Auto-Baixa para 'Realizada':

Se existe relatório criado para o mesmo projeto na mesma data, a visita passa automaticamente para status 'Realizada'.

7.1 Auto-Baixa Automática (Lógica de Negócio)

```
# Para cada visita com status 'Agendada':
vDateStr = v.data_inicio.split('T')[0]
hasMatchingReport = rels.some(r =>
  r.projeto_id === v.projeto_id &&
  r.data_relatorio?.startsWith(vDateStr) &&
  r.status !== 'Rejeitado'
)
if (hasMatchingReport):
  v.status = 'Realizada'
  await saveLocalVisita(v, 'pending')
  await addToSyncQueue('visita', v.id, 'update', ...)
```

8. Módulo Reembolsos e Despesas

Controle de despesas de deslocamento e alimentação vinculadas às obras.

8.1 Fórmula de Cálculo

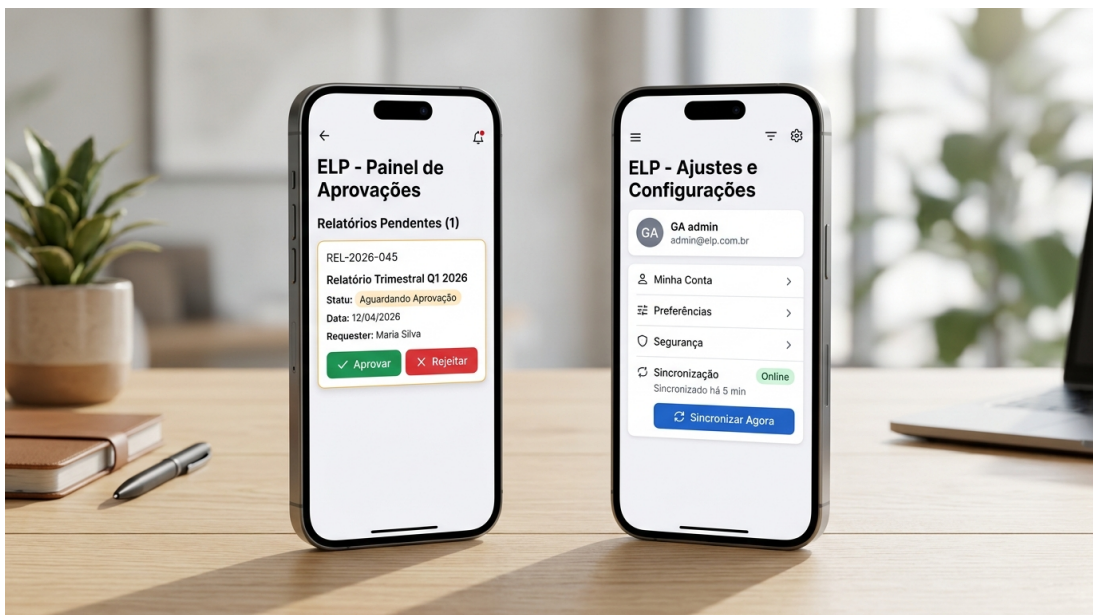
```
total = (quilometragem × valor_por_km) + alimentacao + hospedagem + outros_gastos  
# Exemplo: 120 km × R$ 1,20 = R$ 144,00 + R$ 35,00 alimentação = R$ 179,00
```

8.2 Campos e Status

Campo	Tipo	Default / Observação
Projeto	Dropdown (opcional)	Vincula ao portfólio
Quilometragem	Número decimal	—
Valor por Km (R\$/km)	Número decimal	Default: R\$ 1,20
Alimentação	Número decimal	—
Hospedagem	Número decimal	—
Outros Gastos	Número decimal	—
TOTAL	Calculado automaticamente	Fórmula acima
Status	Enum	Pendente → Aprovado → Pago
Observações	Texto livre	—

9. Painel de Aprovações

Acessível apenas para usuários com `is_master`, `is_aprovador` ou `is_aprovador_express` = true.



À esquerda: Painel de Aprovações (Aprovar / Rejeitar) • À direita: Ajustes & Configurações

9.1 Fluxo de Aprovação

Ação	Detalhamento
✓ Aprovar	Toca em Aprovar → Modal de confirmação → comentário pré-preenchido 'Aprovado em conformidade técnica' → Confirmar → status='Aprovado' → sync POST /api/reports/{id}/approve
✗ Rejeitar	Toca em Rejeitar → Modal exige JUSTIFICATIVA OBRIGATÓRIA → campo vazio bloqueia ação → status='Rejeitado' → sync POST /api/reports/{id}/reject

■ ■ A rejeição sem justificativa exibe Alert 'Justificativa Obrigatória' e bloqueia a ação. O campo de comentário não pode estar vazio.

10. Contatos, Lembretes e Ajustes

10.1 Módulo Contatos

Agenda de contatos técnicos (engenheiros, mestres de obra, clientes) vinculados opcionalmente a obras. Campos: nome, e-mail, telefone, empresa, projeto.

10.2 Módulo Lembretes

Lembretes criados dentro dos formulários de relatório ou na tela dedicada. Cada lembrete tem: texto, projeto vinculado, criado_por, data, flag fechado. Ação 'Fechar lembrete' registra quem fechou e quando. Sincronizados via POST /api/lembretes.

10.3 Ajustes e Configurações

Arquivo: `src/screens/settings/SettingsScreen.tsx` (852 linhas)

Seção	Funcionalidade
Card do Usuário	Avatar, nome, cargo, badge Online/Offline em tempo real
Diagnóstico SQLite	Contadores: N projetos, N visitas, N relatórios, N na fila
Sincronização	Botão 'Sincronizar Agora' (manual) + 'Verificar Atualizações' (OTA)
Legendas Pré-definidas	CRUD completo. Filtro por categoria. Busca textual. Sync Railway.
Preferências	Salvar fotos em álbum dedicado / Alta qualidade (toggles Switch)
Zona de Perigo	'Encerrar Sessão' → limpa AsyncStorage, SQLite permanece no aparelho

10.4 Gestão de Usuários (Admin)

Visível apenas para `is_master=true` ou `username=admin`. Permite criar, editar e desativar usuários do sistema, definir cargo, cor de agenda e permissões de aprovação.

Perfil	Permissões
admin / is_master	Acesso total — Gestão de Usuários, todas as obras e relatórios
is_aprovador	Painel de Aprovações — aprovar e rejeitar relatórios
Engenheiro padrão	Cria e edita apenas seus próprios registros

11. Sistema Offline-First e Sincronização

Arquivo: `src/services/syncService.ts` (358 linhas) — engine central de sincronização bidirecional.

11.1 Três Gatilhos Automáticos

Gatilho	Condição	Debounce
1. Reconexão de Rede	NetInfo detecta isConnected=true	3 segundos
2. Heartbeat Periódico	setInterval a cada 10 segundos	—
3. Retorno do Background	AppState 'active' (foreground)	5 segundos

11.2 Processo de Sync Completo

FASE 1 — Push: Local → Railway

```
# Para cada item na sync_queue (status='pending'):  
1. Marca como 'processing'  
2. Monta payload JSON + anexa fotos em base64 (se relatório)  
3. Executa HTTP (POST/PUT/DELETE) para endpoint correto  
4. Sucesso → marca 'completed', sync_status='synced' na tabela  
5. Falha de rede → mantém 'pending' (retry automático)  
6. Falha HTTP 4xx/5xx → marca 'failed' (não retry automático)  
  
# Caso especial – Relatórios:  
# Após sync, número oficial (REL-2026-047) é gravado de volta no SQLite  
await updateLocalRelatorioNumero(entity_id, officialNumero, syncedAt)
```

FASE 2 — Pull: Railway → SQLite

```
# pullFromServer() faz GET em cada endpoint e salva localmente:  
GET /api/projetos → saveLocalProjeto() para cada registro  
GET /api/visits → saveLocalVisita() para cada registro  
GET /api/relatorios → saveLocalRelatorio() para cada registro  
GET /api/lembretes → saveLocalLembrete()  
GET /api/contatos → saveLocalContato()  
GET /api/reembolsos → saveLocalReembolso()
```

11.3 Tabela sync_queue

Coluna	Tipo	Descrição
id	INTEGER PK AUTOINCREMENT	—
entity_type	TEXT	relatorio, visita, projeto, lembrete, reembolso, contato...
entity_id	INTEGER	ID do registro local afetado

Coluna	Tipo	Descrição
action	TEXT	create update delete approval
endpoint	TEXT	/api/relatorios, /api/visits/123...
method	TEXT	POST PUT DELETE GET
payload	TEXT (JSON)	Objeto serializado completo
status	TEXT	pending processing completed failed
retries	INTEGER	Contador de tentativas
error_message	TEXT	Última mensagem de erro

12. Banco de Dados Local SQLite

Arquivo: `src/database/schema.ts` — 13 tabelas, criadas via `CREATE TABLE IF NOT EXISTS` na inicialização do app.

12.1 Tabelas Principais

Tabela	Linhas Esperadas	Descrição
users	10–50	Usuários e permissões (espelho do backend)
projetos	Ilimitado	Obras cadastradas com especificações de fachada
categorias_obra	~5/obra	Categorias de inspeção por projeto
visitas	Ilimitado	Agenda de visitas técnicas
visita_participantes	N/A	Participantes por visita (N:N)
relatorios	Ilimitado	Relatórios de vistoria padrão
fotos_relatorio	N/A	Fotos com base64 e anotações JSON
relatorios_express	Ilimitado	Relatórios express autônomos
fotos_relatorio_express	N/A	Fotos dos relatórios express
reembolsos	Ilimitado	Solicitações de reembolso de despesas
contatos	Ilimitado	Agenda de contatos técnicos
lembretes	Ilimitado	Lembretes por projeto com status fechado
notificacoes	Ilimitado	Push notifications recebidas
legendas_predefinidas	~50	Banco de legendas para fotos (seed = 10 padrão)
sync_queue	0 (ideal)	Fila de operações HTTP pendentes

12.2 Campo sync_status (todas as tabelas)

Valor	Significado	Ação do Sistema
synced	Sincronizado com Railway	Nenhuma ação pendente
pending	Aguardando envio para nuvem	Será processado no próximo sync
failed	Falha no envio (erro HTTP)	Exige intervenção manual ou retry

13. Sistema de Atualização OTA

O app usa **Expo Updates** para Over-The-Air updates — atualizações sem precisar reinstalar o APK.

13.1 Ciclo de Deploy Automático

```
git push → branch main
↓
GitHub Actions workflow (.github/workflows/)
↓
eas build --platform android --profile preview
↓
ELP.apk publicado nas GitHub Releases
+
expo publish (OTA bundle)
↓
Todos os apps instalados detectam na próxima abertura
e exibem: 'Tem uma atualização recente. Deseja atualizar?'
```

13.2 Build Local

```
# Opção A — Build local rápido (recomendado):
npm run build:apk

# Opção B — EAS Cloud Build:
npx eas-cli build -p android --profile preview

# Resultado: ELP.apk atualizado na raiz do projeto
```

14. Stack Tecnológica Completa

14.1 Mobile App

Biblioteca	Versão	Uso
React Native	via Expo SDK 57	Framework de UI nativo para Android
TypeScript	strict mode	Tipagem estática em todo o projeto
Expo SDK	57	Runtime, APIs nativas e EAS Build
expo-sqlite	v15+	SQLite local (banco de dados no dispositivo)
expo-image-picker	—	Câmera e galeria do dispositivo
expo-file-system	—	Leitura de arquivos locais (base64)
expo-updates	—	Over-The-Air updates automáticos
@react-navigation/native	v7	Sistema de navegação
@react-navigation/native-stack	v7	Stack Navigator (telas empilhadas)
@react-navigation/bottom-tabs	v7	Bottom Tab Bar (5 abas)
@react-native-community/netinfo	—	Deteção de conectividade de rede
@react-native-async-storage	—	Armazenamento chave-valor persistente
axios	—	HTTP client configurado (apiClient)
@expo/vector-icons	—	Ionicons (2000+ ícones)
react-native-safe-area-context	—	Safe area para notch e barras

14.2 Backend (Railway)

Tecnologia	Versão	Uso
Python / Flask	3.1.1	API REST — 609 KB de rotas (routes.py)
PostgreSQL	—	Banco de dados na nuvem (Railway)
SQLAlchemy	2.0.43	ORM Python para modelos de dados
WeasyPrint	66.0	Geração de PDF via HTML+CSS (relatórios)
ReportLab	4.4.3	Geração de PDF programático
Pillow	11.3.0	Processamento de imagens no backend
Resend / SendGrid	—	Envio de e-mails transacionais
OneSignal	—	Push notifications para dispositivos
Google Drive API	—	Backup de relatórios e fotos

Tecnologia	Versão	Uso
APScheduler	—	Tarefas agendadas (scheduler_tasks.py)
Gunicorn	23.0.0	WSGI server de produção

15. Referências de Arquivos do Projeto

Arquivo	Tamanho	Descrição
App.tsx	666 B	Entry point — providers e inicialização
src/navigation/AppNavigator.tsx	6 KB	Stack + Tab navigators, rotas
src/contexts/AuthContext.tsx	5 KB	Contexto de autenticação global
src/contexts/NetworkContext.tsx	2 KB	Estado de rede global
src/database/schema.ts	7 KB	DDL completo das 13 tabelas SQLite
src/database/db.ts	23 KB	Funções CRUD (getLocal*, saveLocal*, etc.)
src/services/syncService.ts	14 KB	Engine de sincronização bidirecional
src/services/api.ts	2 KB	Cliente Axios + URL do servidor
src/services/imageService.ts	10 KB	Câmera, galeria, base64, galeria nativa
src/services/pdfService.ts	7 KB	Geração de PDF local (Express)
src/services/updateService.ts	9 KB	OTA updates via expo-updates
src/screens/dashboard/DashboardScreen.tsx	48 KB	Dashboard (1.346 linhas)
src/screens/reports/ReportFormScreen.tsx	41 KB	Formulário de relatório (1.176 linhas)
src/screens/reports/ReportDetailScreen.tsx	28 KB	Visualização do relatório
src/screens/express/ExpressReportsScreen.tsx	27 KB	Relatório Express (772 linhas)
src/screens/visits/VisitsListScreen.tsx	14 KB	Agenda de visitas (466 linhas)
src/screens/expenses/ExpensesScreen.tsx	14 KB	Reembolsos e despesas
src/screens/approvals/ApprovalDashboardScreen.tsx	10 KB	Painel de aprovações
src/screens/settings/SettingsScreen.tsx	28 KB	Ajustes (852 linhas)
routes.py	609 KB (!)	API backend Flask — todas as rotas da plataforma
models.py	43 KB	Modelos SQLAlchemy do PostgreSQL
pdf_generator_weasy.py	28 KB	Geração de PDF com WeasyPrint
pdf_generator.py	71 KB	Gerador de PDF principal (ReportLab)
notification_service.py	39 KB	Push notifications OneSignal
email_service_unified.py	27 KB	Serviço de e-mail unificado (Resend/SMTP)
ELP.apk	38 MB	Build de produção Android (versão atual)